9

Modelagem de estados

Todos os adultos um dia foram crianças, mas poucos se lembram disso.
– ANTOINE DE SAINT-EXUPÉRY, O PEQUENO PRÍNCIPE

Objetos do mundo real se encontram em estados particulares a cada momento. Por exemplo, uma jarra está *cheia* de líquido; uma pessoa está *cansada*. Da mesma forma, cada objeto participante de um sistema de software orientado a objetos se encontra em um *estado* particular. Um objeto muda de estado quando acontece algum *evento* interno ou externo ao sistema. Quando um objeto muda de um estado para outro, diz-se que ele realizou uma *transição* entre estados. Os estados e as transições de estado de um objeto constituem o seu *ciclo de vida*. Quando da sua transição de um estado para outro, um objeto normalmente realiza determinadas *ações* dentro do sistema. Cada objeto pode passar por um número finito de estados durante a sua vida. Quando um objeto transita de um estado para outro, significa que o sistema no qual ele está inserido também está mudando de estado.

Pela análise das *transições* entre *estados* dos objetos de um sistema de software, podem-se prever todas as possíveis *operações* realizadas, em função de *eventos* que podem ocorrer. O diagrama da UML utilizado para realizar essa análise é o *diagrama de transição de estado* (DTE). Esse diagrama permite descrever o ciclo de vida de objetos de uma classe, os eventos que causam a transição de um estado para outro e a realização de operações resultantes.

A notação e a semântica iniciais do DTE foram propostas por David Harel (Harel, 1987) em seu trabalho com *máquinas de estados finitos* (*finite state ma-*

chines). A idéia básica de Harel foi definir uma máquina com uma quantidade fixa de estados (daí o nome máquina de estados finitos). A máquina pode receber eventos, e cada evento pode ocasionar a transição de um estado para outro. Posteriormente, Jim Rumbaugh (um dos "três amigos") e outros (Mealy e Moore) estenderam a proposta inicial de Harel com várias outras notações e o diagrama de transições de estado foi incorporado à UML.

Na modelagem de sistemas orientados a objetos, a *modelagem dinâmica* descreve o comportamento dinâmico dos objetos durante a execução do sistema. No Capítulo 7, descrevemos dois diagramas da UML para realizar modelagem dinâmica. Utiliza-se também o DTE para obter uma visão dinâmica do sistema. No entanto, diferentemente dos diagramas de interação (que descrevem o comportamento de objetos de classes diferentes), um DTE descreve o comportamento dos objetos de uma única classe. Os diagramas de transição de estado não são definidos para todas as classes de um sistema, mas apenas para aquelas que possuem um número definido de estados conhecidos, e quando o comportamento das classes de objetos é afetado e modificado pelos diferentes estados. Nessas classes, um DTE pode ser utilizado para enfatizar os eventos que resultam em mudanças de estado.

9.1 Diagrama de transição de estado

A UML tem um conjunto rico de notações para desenhar um DTE. Alguns elementos básicos de um diagrama de transição de estados são os *estados* e as *transições*. Associados a estas últimas, estão os conceitos de *evento*, *ação* e *atividade*. Um DTE pode também conter elementos menos utilizados, mas às vezes úteis, como *transições internas*, *estados aninhados* e *estados concorrentes*. A seguir, a notação e a semântica básicas dos diagramas de estado são descritas. Para essa descrição, o DTE da Figura 9-1 é utilizado como exemplo. Esta figura ilustra um DTE para a classe ContaBancária.

9.1.1 Estados

Um *estado* é uma situação na vida de um objeto durante a qual ele satisfaz alguma condição ou realiza alguma atividade. Cada estado de um objeto é normalmente determinado pelos valores dos seus atributos e (ou) pelas suas ligações com outros objetos. Por exemplo: "o atributo *reservado* deste objeto livro tem valor *verdadeiro*". Outro exemplo: "uma conta bancária passa para o *vermelho* quando o seu saldo fica *negativo*".

A notação UML para representar um estado é um retângulo com as bordas arredondadas. Na sua forma mais simples, o retângulo de um estado possui um único compartimento (conforme ilustrado na Figura 9-2). O estado inicial é re-

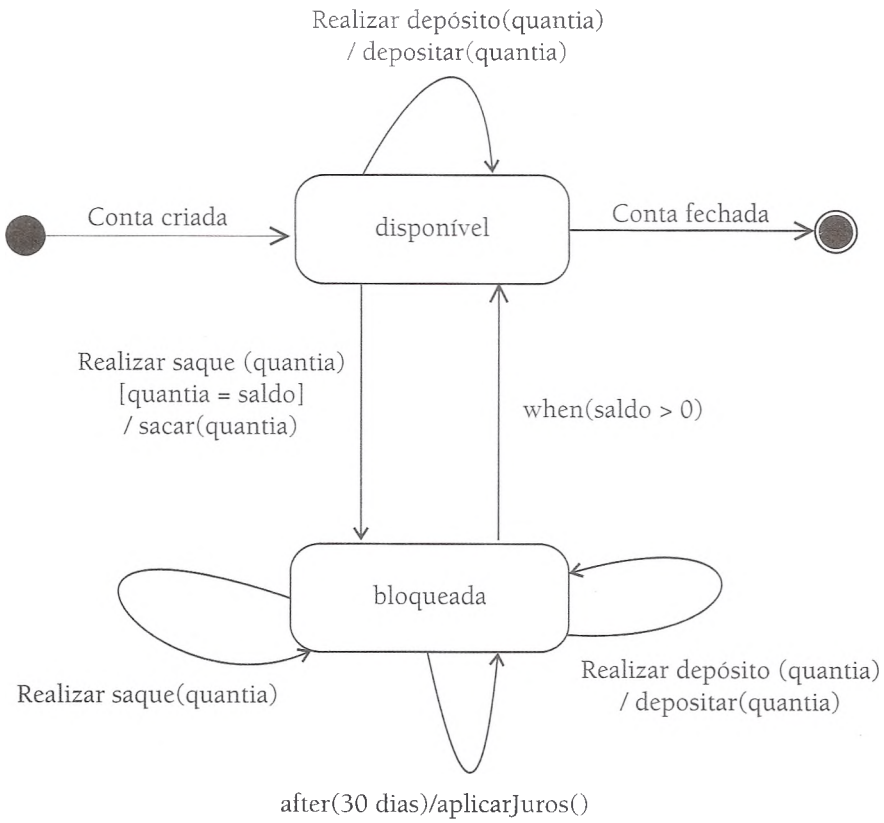


Figura 9-1: Exemplo de DTE para a classe ContaBancária.

presentado como um círculo preenchido e indica o estado de um objeto quando ele é criado. Só pode haver um estado inicial em um DTE. Essa restrição serve para definir a partir de que ponto um DTE deve começar a ser lido.[1] O estado final é representado como um círculo "eclipsado" (ver Figura 9-2) e indica o fim do ciclo de vida de um objeto. Este estado é opcional e pode haver mais de um estado final em um DTE.

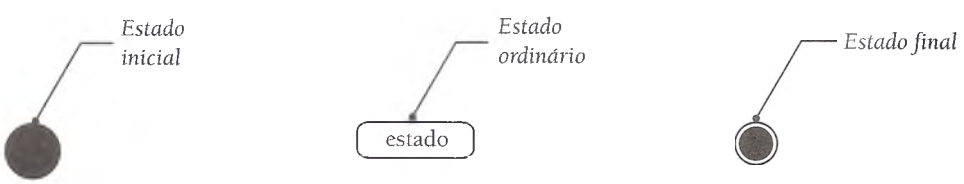


Figura 9-2: Elementos gráficos básicos de um DTE.

[1] Na verdade, conforme descrito mais adiante, em um DTE com estados aninhados, pode haver mais de um estado inicial.

9.1.2 Transições

Os estados estão associados a outros pelas *transições*. Uma transição é mostrada como uma linha conectando estados, com uma seta apontando para um dos estados. Quando ocorre uma transição entre estados, diz-se que a transição foi *disparada*. Note que, em uma transição, o estado subseqüente pode ser igual ao estado original. Uma transição pode ser rotulada com uma expressão. A seguir, é apresentada a forma geral dessa expressão. Seus detalhes são descritos nas seções a seguir.

```
evento (lista-parâmetros) [guarda] / ação
```

9.1.3 Eventos

Uma transição possui um *evento* associado. Um evento é algo que acontece em algum ponto no tempo e que *pode* modificar o estado de um objeto. Alguns exemplos de eventos são listados a seguir:

Tabela 9-1: Exemplos de nomes de eventos em sistemas de software e em processos de negócio

Em sistemas de software	Em processos de negócio
1 Mouse pressionado	1 Pedido realizado
2 Disco inserido	2 Fatura paga
	3 Cheque devolvido

Um evento pode conter uma *lista de parâmetros* (veja o elemento lista-parâmetros na sintaxe anterior). Os parâmetros são passados para fornecer informações úteis ao objeto receptor de evento. A lista de parâmetros é especificada entre os parâmetros. Por exemplo, na expressão `Mouse Pressionado (local)`, o parâmetro local indica em que posição da tela o mouse foi pressionado.

Os eventos relevantes a uma classe de objetos podem ser classificados em três tipos. Estes tipos são descritos a seguir.

1. **Evento de chamada:** *recebimento de uma mensagem de outro objeto*. Pode-se pensar neste tipo de evento como uma solicitação de serviço de um objeto a outro.
2. **Evento de sinal:** *recebimento de um sinal*. Este é um tipo especial do evento anterior. Neste evento, o objeto recebe um sinal de outro objeto que pode fazê-lo mudar de estado. A diferença básica entre o evento de sinal e o evento de chamada é que neste último o objeto que envia a mensagem fica esperando a execução da mesma. No evento de sinal, o objeto reme-

tente continua o seu processamento após ter enviado o sinal. O evento de sinal é raramente utilizado.

3. **Evento temporal:** *passagem de um intervalo de tempo predefinido.* Em vez de receber uma mensagem específica, um objeto pode interpretar a passagem de um certo intervalo de tempo como sendo um evento.[2] Um evento temporal é especificado com a cláusula *after* seguida de um parâmetro que especifica um intervalo de tempo. Por exemplo, a expressão `after(30 segundos)` indica que a transição correspondente será disparada 30 segundos após o objeto ter entrado no estado atual.
4. **Evento de mudança:** *uma condição que se torna verdadeira.* O fato de uma determinada condição se tornar verdadeira também pode ser visto como um evento. Este evento é representado por uma expressão de valor lógico (*verdadeiro* ou *falso*) e é especificado utilizando-se a cláusula `when`. Por exemplo, a expressão `when(saldo > 0)` significa que a transição é disparada quando o valor do atributo *saldo* for positivo (ver Figura 9-1). Eventos temporais (descritos no item anterior) também podem ser definidos utilizando-se a cláusula when. Alguns exemplos: `when(data = 13/07/2002)`, `when(horário = 00:00h)` etc.

Considere a Figura 9-1. Há uma *transição reflexiva* (ou seja, uma transição que sai de um estado e entra no mesmo estado) quando a conta bancária está `bloqueada`. Essa transição possui um evento temporal, indicando que 30 dias após a conta ter entrado nesse estado, a operação `aplicarJuros()` é executada. A Figura 9-1 também apresenta um exemplo de evento de mudança na transição de *bloqueada* para *disponível*. Esse evento passa um parâmetro (*saldo* > *0*) correspondente a uma expressão condicional. Quando essa expressão for verdadeira, a transição é disparada.

A ocorrência de um evento relevante pode ocasionar outro evento. A Figura 9-3 ilustra esquematicamente dois diagramas de estados. O primeiro exibe a forma normal de representação de um evento: quando `Evento` ocorre, o objeto

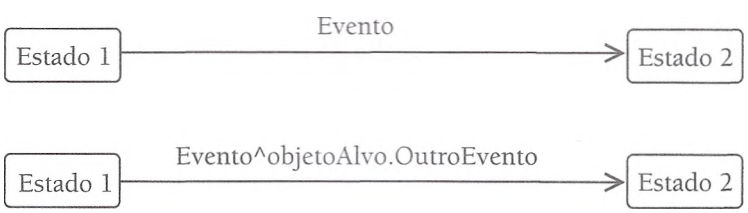


Figura 9-3: Ocorrência de evento ativando outro evento.

[2] Para o leitor familiarizado com a terminologia da Análise Estruturada, esse evento é equivalente ao *evento temporal.*

passa de `Estado 1` para `Estado 2`. A mesma transição acontece no segundo DTE. Entretanto, além da transição de estados, o evento `OutroEvento` (relevante a `objetoAlvo`) é disparado.

9.1.4 Condição de guarda

Uma transição pode também ter uma *condição de guarda*. Uma *condição de guarda*, ou simplesmente *guarda*, é uma expressão de valor lógico, da mesma forma que a utilizada para diagramas de interação (ver Seção 7.1.1.2.2). A condição de guarda de uma transição pode ser definida utilizando-se parâmetros passados no evento e também atributos e referências a ligações da classe em questão. Além disso, uma condição também pode testar o valor de um estado.

Uma transição na qual foi definida uma condição de guarda é disparada somente se o evento associado ocorre *e* a condição de guarda é verdadeira. Se uma transição não tiver uma condição de guarda, ela sempre será disparada quando o evento ocorrer. É importante não confundir a condição de guarda de uma transição com um evento de mudança (que também é definido com uma expressão de valor lógico). A expressão condicional de uma condição de guarda é sempre apresentada entre colchetes. Ao contrário, a expressão condicional nos eventos de mudança são parâmetros da cláusula `when`.

Como exemplo, a Figura 9-1 ilustra uma transição entre os estados *disponível* e *bloqueada*. Essa expressão possui uma condição de guarda, [`quantia = saldo`], onde quantia é um parâmetro recebido e `saldo` é um atributo da classe `ContaBancária`. Quando o evento `Realizar saque` ocorre, a transição somente ocorrerá se a condição de guarda for verdadeira. Se não for o caso, o evento é ignorado pelo objeto.

9.1.5 Ações

Ao transitar de um estado para outro, um objeto pode realizar uma ou mais ações. Uma ação é uma expressão que pode ser definida em termo dos atributos, das operações ou das associações da classe. Os parâmetros do evento também podem ser utilizados. Uma ação pode também corresponder à execução de uma operação. A ação é representada na linha da transição e deve ser precedida por uma barra inclinada para a direita (símbolo "/"). Note que a ação associada a uma transição é executada somente se a transição for disparada. Uma ação pode gerar outros eventos. Esses eventos podem ser relevantes para o próprio objeto ou para outros objetos.

Pode haver ações especiais que são executadas sempre que um objeto passa para um determinado estado (não importa por qual transição) ou sempre que o objeto sai de um estado (também não importando por qual transição). Essas ações especiais são descritas na Seção 9.1.8.

9.1.6 Atividades

Semelhante a uma ação, uma atividade é algo que é executado pelo objeto. No entanto, uma atividade pode ser interrompida (considera-se que o tempo de execução de uma ação é tão insignificante que esta não pode ser interrompida). Por exemplo, enquanto a atividade estiver em execução, pode acontecer um evento que a interrompa e cause uma mudança de estado. Uma outra diferença entre ação e atividade é que uma atividade sempre está associada a um estado (ao contrário, uma ação está associada a uma transição). A forma de declarar atividades é descrita na Seção 9.1.8.

9.1.7 Ponto de junção

Em algumas situações, o próximo estado de um objeto varia de acordo com o valor da condição de guarda. Se o valor da condição de guarda for verdadeiro, o objeto vai para um estado E1; se o valor for falso, o objeto vai para outro estado E2. É como se a transição tivesse bifurcações, e cada transição de saída da bifurcação tivesse uma condição de guarda. Essa situação pode ser representada em um DTE por um *ponto de junção*.

Um ponto de junção é desenhado como um losango[3] em que chegam uma ou mais transições (provenientes de estados diferentes) e de onde partem uma ou mais transições. A cada transição de saída do ponto de junção está associada uma condição de guarda. A transição que o objeto segue é aquela para a qual a condição de guarda é verdadeira. A Figura 9-4 apresenta de forma esquemática um fragmento de DTE que ilustra a utilização de um ponto de junção. Desse ponto, partem quatro transições. Cada uma dessas transições está associada a uma condição de guarda e a uma ação. As ações são opcionais. É importante enfatizar que, quando uma das condições é verdadeira, todas as demais são falsas.

Pontos de junção permitem que duas ou mais transições compartilhem uma "trajetória de transições". Na Figura 9-4, por exemplo, as transições que saem dos estados `E0` e `E1` compartilham o mesmo caminho de transições a partir do ponto de junção. De uma forma geral, pode haver um número ilimitado de transições saindo de um ponto de junção. Além disso, nada impede que o estado no qual chega uma transição que saiu de um ponto de junção seja o mesmo do qual saiu uma transição que chegou a tal ponto de junção. Pode haver também uma transição de saída que esteja rotulada com a cláusula `else`. Isso significa que, se todas as outras condições de guarda forem falsas, a transição correspondente à clausula `else` será disparada.

[3] Uma notação alternativa é utilizar um pequeno círculo, semelhante à representação de um estado inicial.

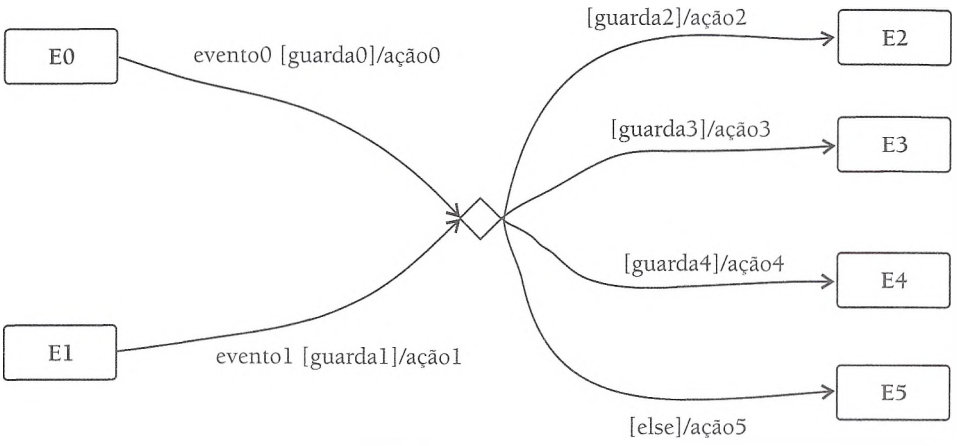


Figura 9-4: Utilização de pontos de junção em um DTE.

9.1.8 Cláusulas entry, exit e do

No compartimento adicional de um retângulo de estado, podem-se especificar ações ou atividades a serem executadas.

1. A cláusula `entry` pode ser usada para especificar uma ação a ser realizada no momento em que o objeto entra em um estado. A ação dessa cláusula é sempre executada, independentemente do estado do qual o objeto veio (é como se a ação especificada estivesse associada a todas as transições de entrada no estado).
2. A cláusula `exit` serve para declarar ações que são executadas sempre que o objeto sai de um estado. Da mesma forma que a cláusula `entry`, a ação da cláusula `exit` é sempre executada, independentemente do estado para o qual o objeto vai (é como se a ação especificada estivesse associada a todas as transições de saída do estado).
3. A cláusula `do` serve para definir alguma atividade a ser executada quando o objeto passa para um determinado estado. (Note a diferença em relação à cláusula `entry`, que serve para especificar uma ação em vez de uma atividade.)

As cláusulas predefinidas (`entry`, `exit`, `do`) são especificadas no interior do retângulo do estado e têm a seguinte sintaxe:

```
evento / [ação | atividade]
```

A Figura 9-5 ilustra de forma esquemática um DTE no qual foi especificada a cláusula `do`. Note que a inexistência de um evento na transição de `Estado 2` para `Estado 3` indica que, assim que a `atividade2` for finalizada, a transição ocorrerá

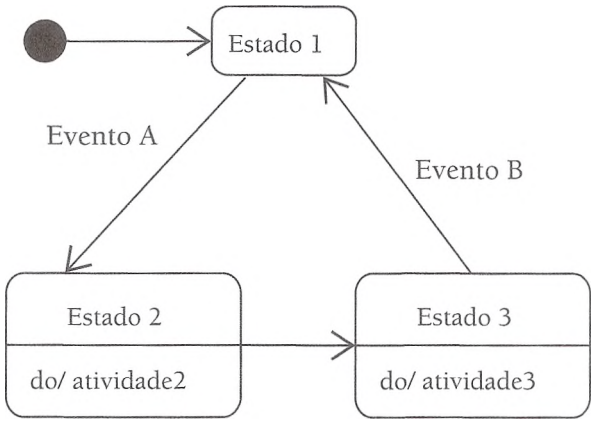


Figura 9-5: Exemplo de utilização da cláusula *do*.

(a transição é automática). Por outro lado, a transição de `Estado 3` para `Estado 1` pode ocorrer antes de a `atividade3` terminar, bastando para isso que o `Evento B` ocorra antes do término da atividade.

A Figura 9-6, adaptada da especificação da UML (OMG, 2001), ilustra um estado no qual são declaradas as cláusulas `entry` e `exit`. A leitura desse estado indica que a ação `definirEco(cInvisivel)` é executada sempre que o objeto (provavelmente um objeto de fronteira) entra nesse estado. O mesmo acontece com a ação `definirEco(cVisivel)` quando o objeto sai do estado.

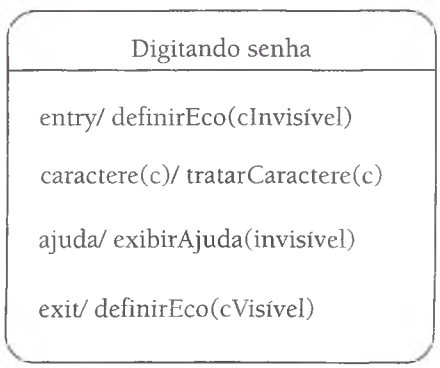
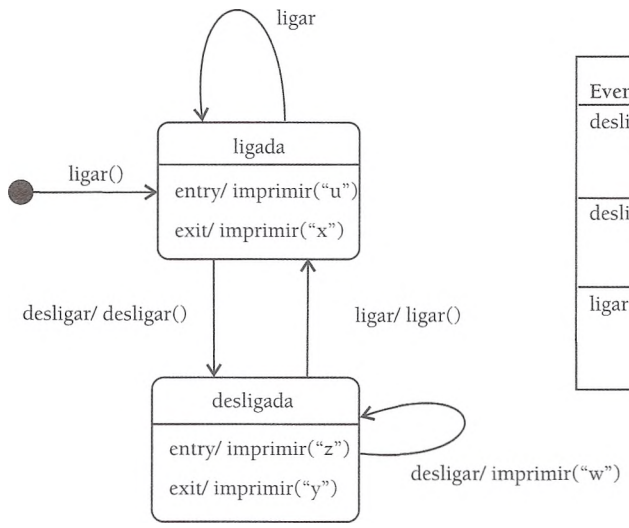


Figura 9-6: Exemplo de estado no qual foram definidas as cláusulas *entry*, *exit* e transições internas.

A Figura 9-7 é outro exemplo que ilustra o uso das ações `entry` e `exit`. O diagrama de estados apresentado representa os estados de uma lâmpada (*ligada* ou *desligada*) e os eventos relevantes (*ligar* e *desligar*). À direita desta figura, é apresentada a ordem de execução das operações quando a lâmpada está *ligada*, e os seguintes eventos ocorrem sucessivamente: *desligar*, *desligar*, *ligar*.



Evento	Ações executadas
desligar	imprimir("x") desligar() imprimir("z")
desligar	imprimir("y") imprimir("w") imprimir("z")
ligar	imprimir("y") ligar() imprimir("u")

Figura 9-7: Utilização das cláusulas *entry* e *exit*.

9.1.9 Transições internas

Além das ações predefinidas `entry` e `exit`, zero ou mais *transições internas* podem também ser especificadas. Uma transição interna é uma transição que não faz o objeto mudar de estado. O objetivo das transições internas é fazer com que uma ação ou atividade seja executada sem que uma mudança de estados ocorra. A sintaxe de uma transição interna é a mesma utilizada para cláusulas `entry`, `exit` e `do`. Como exemplo, considere novamente a Figura 9-6. No DTE dessa figura, há duas transições internas: *caractere* e *ajuda*. Quando algum dos eventos associados a essas transições ocorre, a ação correspondente é executada, mas o objeto permanece no estado `Digitando senha`.

De uma forma geral, a declaração das cláusulas `entry`, `exit` e `do`, e das transições internas, pode ser em qualquer ordem dentro do retângulo do estado. No entanto, recomenda-se utilizar a seguinte ordem de declaração: cláusula `entry`, cláusula `do`, transições internas e cláusula `exit`.

Um ponto importante é a ordem na qual as ações predefinidas e as transições internas são executadas. Ações de saída (`exit`) precedem ações externas associadas a uma transição para fora do estado. Ações de entrada (`entry`) sucedem ações externas associadas a uma transição para o estado. Em uma transição reflexiva, a ordem de execução é a seguinte: ação `exit`, ação da transição reflexiva, ação `entry` e, por fim, as demais ações ou atividades.

9.1.10 Exemplo

Como um exemplo mais completo dos elementos básicos de um DTE aqui descritos, considere a Figura 9-8, um DTE para um relógio despertador, que apre-

senta um exemplo de utilização da cláusula *do*. Esse despertador funciona da seguinte forma: quando o evento `Armar alarme` ocorre, o parâmetro horário do evento informa o horário no qual o despertador deve tocar o alarme. O despertador fica esperando até que esse horário chegue. Isso ocorre quando a expressão de valor lógico definida no evento temporal `when(horário Definido = agora())` se torna verdadeira.[4] A ocorrência desse evento acarreta a transição de `esperando` para `tocando alarme`. O estado `tocando alarme` possui uma cláusula `entry`, na qual uma operação, `tocarAlarme`, é executada. Essa operação toca o alarme por 10 segundos. Agora, note a transição de `tocando alarme` para `esperando`. Essa transição não possui nem evento nem condição de guarda. De fato, essa transição é automaticamente disparada quando a ação especificada na cláusula `entry` é finalizada. Isso serve para exemplificar que, de uma forma geral, uma associação que não possui um evento associado é disparada logo após as ações internas ao estado tiverem sido executadas. A ação associada a essa transição é executada, fazendo com que o novo horário de alarme seja definido (o alarme toca durante 10 segundos a cada 5 minutos).[5]

Note que o estado `esperando` possui uma cláusula `entry` que incrementa um contador (*n* é um atributo privativo da classe `Despertador`). Esse contador serve para definir quantas vezes o alarme será tocado. Neste exemplo, o alarme será tocado 3 vezes, se não for desarmado antes disso. A qualquer momento, pode ocorrer um evento Desarmar alarme, que faz com que o alarme seja desarmado.

9.1.11 Estados aninhados

Podem existir estados aninhados dentro de um outro estado. Um estado que contém diversos outros é dito *composto*. Todos os estados dentro de um estado composto herdam qualquer transição deste último. Isso significa que o uso de estados compostos geralmente torna um DTE mais legível.

A Figura 9-9 apresenta um exemplo de DTE com um estado composto, `armado`. Esse estado possui dois estados aninhados, `esperando` e `tocando alarme`. Compare com o DTE equivalente, porém menos legível, da Figura 9-8. Quando ocorre a transição de `desarmado` para `armado`, o estado aninhado `esperando` é ativado. A qualquer momento em que ocorrer o evento de chamada `Desarmar` alarme ou o evento temporal `when(n>3)`, não importa em que estado aninhado o objeto esteja, o novo estado será `desarmado`.

---

[4] Considere que a função `agora( )` informe o horário corrente. Essa função pode ser uma operação da classe ou uma função do sistema operacional.

[5] Na verdade, uma transição não precisa ter ação ou um evento associado. Nessa situação, a transição é disparada quando as ações internas ao estado terminam. Essas transições são denominadas *transições automáticas*. Se houver várias transições automáticas partindo de um estado, obrigatoriamente cada uma deve ter uma condição de guarda, e as condições devem ser mutuamente exclusivas.

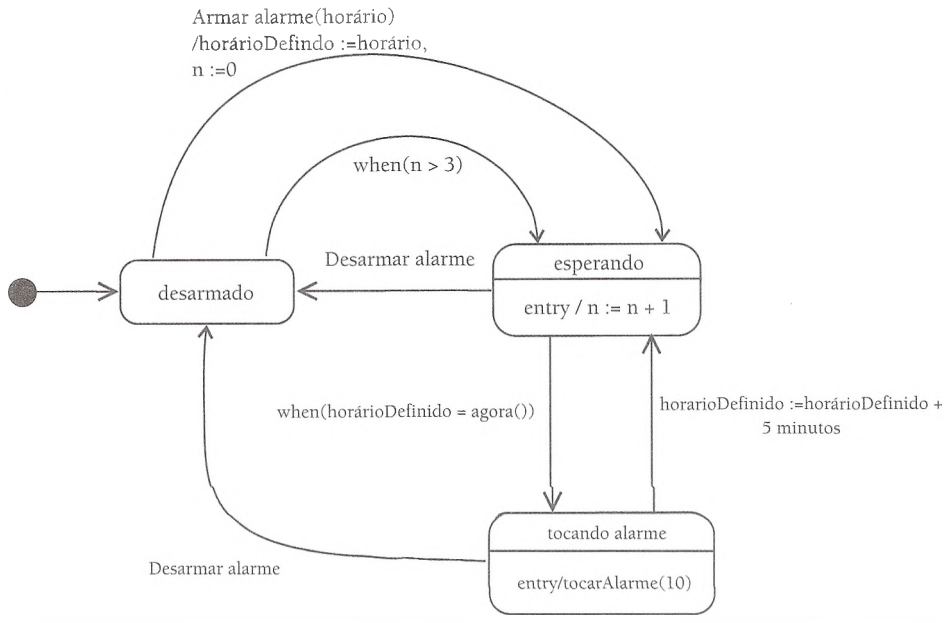


Figura 9-8: Diagrama de estados para a classe *Despertador*.

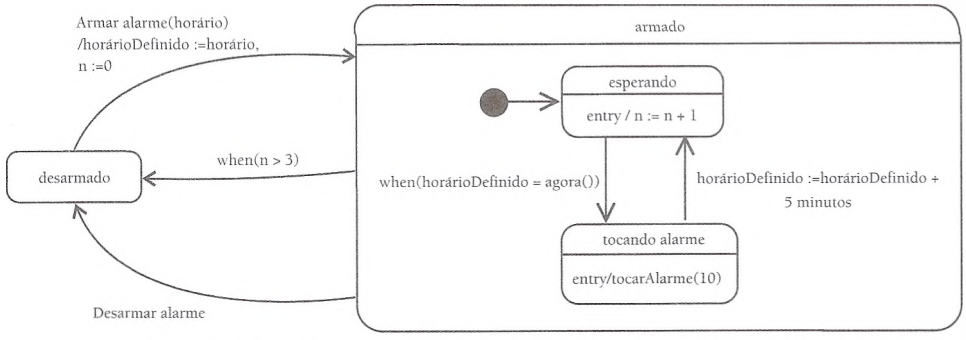


Figura 9-9: Exemplo de DTE com um estado composto (*armado*).

9.1.12 Estados concorrentes

Um estado concorrente é um tipo especial de estado composto. Um objeto em um estado concorrente pode, na verdade, se encontrar em dois ou mais estados independentes.

O diagrama de estados da Figura 9-10 ilustra dois conjuntos de estados independentes da classe `Refrigerador`. Um objeto dessa classe pode estar com a porta aberta ou fechada. Independentemente, esse objeto pode estar com o motor ligado ou desligado. O estado composto torna mais simples a representação desses dois conjuntos de estados. Sem um estado composto, haveria um total de transições igual à cardinalidade do produto cartesiano dos dois conjuntos de estados.

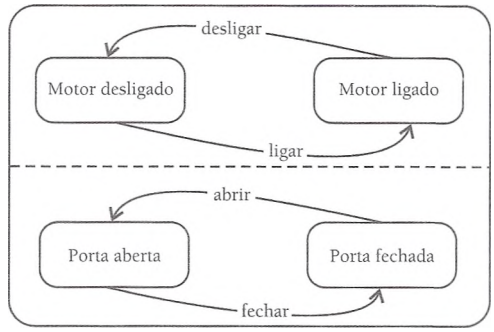


Figura 9-10: Estado composto para a classe Refrigerador.

9.2 Identificação dos elementos de um diagrama de estados

Os estados podem ser vistos como uma abstração dos atributos e associações de um objeto. A seguir, temos alguns exemplos (os nomes em itálico são possíveis estados).

- Um professor está *licenciado* quando não está ministrando curso algum durante o semestre.
- Um tanque está *na reserva* quando o valor do nível de combustível está abaixo de 20%.
- Um pedido está *atendido* quando todos os seus itens estão atendidos.

Um bom ponto de partida para identificar estados de um objeto é analisar os possíveis valores de seus atributos e as ligações que ele pode realizar com outros objetos. No entanto, a existência de atributos ou ligações não é suficiente para justificar a criação de um DTE para uma classe. O comportamento de objetos dessa classe deve depender de tais atributos ou ligações.

Em relação a transições, devemos identificar os eventos que podem gerá-las. Além disso, deve-se examinar também se há algum fator que condicione o disparo da transição. Se existir, esse fator deve ser modelado como uma condição de guarda da transição. Já que as transições dependem de eventos para ocorrer, devem-se identificar esses eventos primeiramente. Um bom ponto de partida para identificar esses eventos é a descrição dos casos de uso.

Os eventos encontrados na descrição dos casos de uso são externos ao sistema. Contudo, uma transição pode também ser disparada por um evento *interno* ao sistema (ver Seção 9.1.2). Para identificar os eventos internos relevantes a um objeto, analise os diagramas de interação. Esses diagramas contêm mensagens sendo trocadas entre objetos, e mensagens nada mais são do que solicitações de um objeto a outro. Essas solicitações podem ser vistas como eventos. De uma forma geral, cada operação com visibilidade pública de uma classe pode ser vista como um evento em potencial.

Outra fonte para identificação de eventos relevantes é analisar as regras de negócio definidas para o sistema (ver Seção 4.5.1). Normalmente essas regras possuem informações sobre condições limites que permitem identificar estados e transições. Por exemplo, vamos analisar alguns exemplos de regras de negócio.

- "Um cliente do banco não pode retirar mais de R$1.000 por dia de sua conta". Possivelmente, há dois estados para uma conta bancária: *disponível* e *bloqueada*. Além disso, a transição de um estado para outro depende do atributo *saldo* (por exemplo) da conta bancária.
- *"Os pedidos para um cliente não especial devem ser pagos antecipadamente"*. O cliente pode estar no estado de cliente especial e cliente comum.
- *"O número máximo de alunos por curso é igual a 30"*. Se houver uma classe `Curso` e uma classe *Aluno* no diagrama de classes, essa regra de negócio indica que os estados (*aberto* ou *fechado*) de um objeto `Curso` dependem da quantidade de ligações desse objeto com objetos de Aluno.

9.3 Construção de diagramas de transições de estados

Para sistemas bastante simples, a definição dos estados de todos os objetos não é tão trabalhosa. No entanto, a quantidade de estados possíveis de todos os objetos de um sistema complexo é grande. Isso torna a construção de um DTE para o sistema todo uma tarefa bastante complexa.

Por exemplo, considere, sem perda de generalidade, que todos os objetos de um sistema têm o mesmo número de estados: três estados cada um. Para um sistema de dois objetos, a quantidade de estados possíveis do sistema é 9 (3 x 3). Para um sistema de 4 objetos, a quantidade de estados possíveis do sistema passa para 81 (3 x 3 x 3 x 3). Ou seja, o crescimento dos estados de um sistema se dá exponencialmente com o número de objetos, o que torna impraticável a construção de um único diagrama para todo o sistema.

Para resolver o problema da explosão exponencial de estados, os diagramas de estados são desenhados por classe. Geralmente, cada classe é simples o suficiente para que o diagrama de estados correspondente seja compreensível. Note, contudo, que essa solução de dividir a modelagem de estados por classes do sistema tem a desvantagem de dificultar a visualização do estado do sistema como um todo. Essa desvantagem é parcialmente compensada pela construção de diagramas de interação (Capítulo 7).

Nem todas as classes de um sistema precisam de um DTE. Um diagrama de estados é desenhado somente para classes que exibem um comportamento dinâmico relevante. A relevância do comportamento de uma classe depende muito de cada situação em particular. No entanto, objetos cujo histórico precisa ser rastreado pelo sistema são objetos típicos para se construir

um diagrama de estados. Por exemplo, considere um objeto *Pedido*. Esse objeto é criado quando o cliente realiza um pedido. O crédito deve ser aprovado para que o pedido seja *aceito*. Se o crédito é negado, o pedido é retornado ao cliente para ser modificado. Se o crédito é aceito, o pedido é *confirmado*. Após ser atendido (ou seja, todos os itens desse pedido foram providenciados), esse pedido é enviado ao sistema de distribuição para ser *entregue*. Depois disso, por alguma razão, o pedido pode ser *devolvido* pelo cliente. Esse histórico de estados de um pedido é relevante para o sistema (por exemplo, só os pedidos atendidos podem ser entregues aos clientes). Esse é um caso típico de classe cuja definição de um diagrama de estados ajudaria a esclarecer o seu comportamento.

Um objeto de entidade pode perdurar durante várias execuções do sistema. Conseqüentemente, esse objeto pode participar da realização de diversos casos de uso do sistema. Diagramas de estados são úteis para capturar o comportamento de *um* objeto de entidade durante *vários* casos de uso. Entretanto, note que não apenas as classes de entidade são candidatas a precisarem de um DTE. As classes de controle e classes da fronteira (ver Seção 5.4.2) freqüentemente precisam de diagramas de estados para melhor esclarecer o seu comportamento.

Uma vez selecionadas as classes para cada uma das quais se deve construir um diagrama de estados, acompanhe esta seqüência de passos.

1. Identifique os estados relevantes para a classe.
2. Identifique os eventos relevantes aos objetos de uma classe. Para cada evento, identifique qual transição que ele ocasiona.
3. Para cada estado, identifique as transições possíveis quando um evento relevante ocorre.
4. Para cada estado, identifique os eventos internos e as ações correspondentes relevantes.
5. Para cada transição, verifique se há fatores que influenciam o seu disparo. Se esse for o caso, uma condição de guarda deve ser definida. Verifique também se alguma ação deve ser executada quando uma transição é disparada (ações).
6. Para cada condição de guarda e para cada ação, identifique os atributos e as ligações que estão envolvidos. Pode ser que esses atributos ou ligações ainda não existam. Nesse caso, eles devem ser adicionados ao modelo de classes.
7. Defina o estado inicial e os eventuais estados finais.
8. Desenhe o diagrama de estados. Procure posicionar os estados de tal forma que o ciclo de vida do objeto possa ser visualizado de cima para baixo. e da esquerda para a direita.

É importante notar que a construção de diagramas de estados de um sistema freqüentemente leva à descoberta de novos atributos para uma classe, principalmente atributos que servem de abstrações para estados. Além disso, esse processo de construção permite identificar novas operações na classe, pois os objetos precisam reagir aos eventos que eles recebem, e essas reações são realizadas mediante operações.

9.4 Modelagem de estados no processo de desenvolvimento

A modelagem de estados é uma atividade que deve começar na fase de análise, em paralelo à construção nos diagramas de interação e nos diagramas de classes, e utilizando informações geradas nessa construção, os diagramas de estados podem ser construídos para as classes cujos estados são considerados relevantes.

O diagrama de classes fornece informações para a construção do diagrama de estados (ver Seção 9.2), pois é no primeiro que as classes com estados relevantes são identificadas. Por outro lado, durante a construção do diagrama de estados para uma classe, novos atributos e novas operações podem surgir. Essas novas propriedades devem ser adicionadas ao modelo de classes. Dessa forma, assim como vimos para a modelagem de interações, a modelagem de estados é uma atividade que tem a característica de retroalimentar as atividades de modelagem das quais obtém informações.

Deve-se notar também que o comportamento de um objeto varia em função do estado no qual ele se encontra. Em vista disso, pode haver a necessidade de atualizar a descrição de um ou mais métodos da classe desse objeto para indicar como as operações correspondentes se comportam em função de cada estado. Por exemplo, o comportamento da operação sacar( ) da classe `ContaBancária` (Figura 9-1) varia em função do estado no qual essa classe se encontra (saques não podem ser realizados em uma conta bloqueada).

Outra característica representada em um DTE e que resulta em alterações no modelo de classes é o *estado inicial* do objeto. Isso porque o valor do estado inicial normalmente é identificado durante a modelagem dos estados da classe. O método de instanciação (construtor) da classe em questão deve definir o esta-

```
public class Lâmpada
{
        .....
        private String estado;
        public Lâmpada () { estado = "desligada";}
        .....
}
```

Figura 9-11: O estado inicial é definido na operação de instanciação.

do inicial de um objeto. Normalmente, informações para definição do estado inicial de um objeto são passadas como argumentos para o construtor. Por exemplo, a Figura 9-11 ilustra um fragmento de declaração da classe `Lâmpada` em linguagem Java. Essa classe apresenta um método de instanciação que define o valor do atributo `estado`, responsável por manter a informação do estado atual do objeto.

É importante também notar a diferença de objetivo com o uso do diagrama de interações e do diagrama de transições de estados. Enquanto um diagrama de estados representa o comportamento de *um objeto*, os diagramas de interação (estudados no Capítulo 7) são utilizados para capturar o comportamento de *vários objetos* em um (cenário de) caso de uso. No entanto, para capturar o comportamento de vários objetos em vários casos de uso, a melhor alternativa são os diagramas de atividade, descritos no Capítulo 10.

9.5 Estudo de caso

Esta seção continua a modelagem do Sistema de Controle Acadêmico. Das classes desse sistema, uma com certeza precisa de um DTE: `OfertaDisciplina`. Foram identificados os seguintes estados para essa classe:

Aberta	A oferta de disciplina está aberta para receber inscrições de alunos, até a sua quantidade máxima. O professor, as salas e horários foram alocados.
Lotada	A oferta de disciplina alcançou a sua capacidade máxima em relação à quantidade de alunos inscritos.
Fechada	A oferta de disciplina está totalmente definida (os alunos estão inscritos).
Cancelada	A oferta de disciplina é cancelada. O evento de cancelamento pode acontecer a qualquer momento do ciclo de vida de uma oferta de disciplina.

Em relação ao estado `Lotada`, o leitor deve se lembrar de que há uma regra de negócio do sistema que define a quantidade máxima de alunos. Essa regra tem identificador `RN02` e sua declaração se encontra na Seção 4.7.2.

Os eventos que são relevantes para objetos da classe `OfertaDisciplina` são a inscrição de um aluno, assim como as operações que realizam abertura, cancelamento e fechamento de uma oferta de disciplina. Além disso, o atributo que armazena a quantidade atual de alunos inscritos deve ser definido para monitorar quando essa quantidade atingir o valor máximo.

O DTE para a classe `OfertaDisciplina` está ilustrado na Figura 9-2.

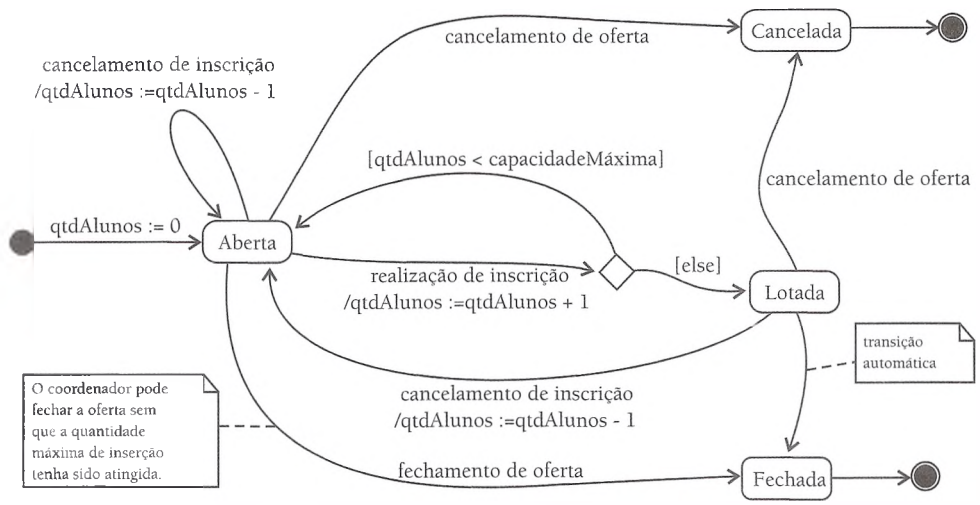


Figura 9-12: DTE para a classe OfertaDisciplina.

Conforme descrito na Seção 9.4, a construção de um DTE para uma classe pode resultar na definição de novas propriedades nessa classe. De fato, a classe `OfertaDisciplina` deve ser atualizada com as informações obtidas na construção de seu DTE. A Figura 9-13 ilustra a classe atualizada com base nas informações obtidas a partir de seu DTE.

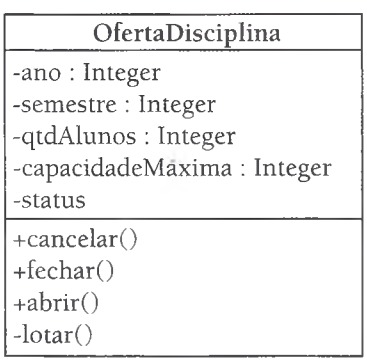


Figura 9-13: Definição atualizada da classe `OfertaDisciplina`, após a construção de seu DTE.

EXERCÍCIOS

9-1: Descreva a posição do diagrama de estados no processo de desenvolvimento incremental e iterativo. Quando eles são utilizados? Para que são utilizados?

9-2: Modele através de um diagrama de estados a seguinte situação: em uma máquina de encher garrafas de refrigerante passam diversas garrafas. Uma garrafa entra inicialmente vazia no equipamento. A partir de um determinado momento, ela começa a ser preenchida com refrigerante. Ela permanece nesse estado (sendo preenchida) até que, eventualmente, esteja cheia de refrigerante. Nesse momento, o equipamento sela a garrafa com uma tampinha, e assim a garrafa passa para o estado de "lacrada". Uma garrafa vazia não deve ser lacrada pelo equipamento. Além disso, uma garrafa cheia de refrigerante não deve receber mais esse líquido.

9-3: Construa um diagrama de estados considerando o seguinte "ciclo de vida" de um paciente de hospital. O paciente entra no hospital, vítima de um acidente de carro. Ele é encaminhado para a emergência. Após uma bateria de exames, esse paciente é operado. Alguns dias depois, o paciente é movido da grande emergência do hospital para a enfermaria, pois não corre mais perigo de vida. Depois de passar por um período de observação na enfermaria, o paciente recebe alta médica.

9-4: Construa um DTE para a classe Pedido, mencionada na Seção 9.3.

9-5: Construa um diagrama de estados para uma classe Mensagem, que representa uma mensagem de correio eletrônico. Como dica, considere os estados apresentados a seguir.

a. Recebida: este é o estado inicial. A mensagem acabou de entrar na caixa de correio e permanece nesse estado até ser lida.
b. Lida: a mensagem é lida pelo usuário.
c. Respondida: o usuário responde à mensagem.
d. Na lixeira: usuário remove a mensagem da caixa de correio.

9-6: Construa um diagrama de estados para um aparelho de secretária eletrônica. Como dica, considere os estados apresentados a seguir. Utilize estados compostos se for necessário.

a. *Registrando recados*: alguém faz uma chamada para o aparelho de telefone ao qual a secretária está conectada, e a chamada não é atendida. A secretária, então, apresenta a mensagem gravada pelo usuário do aparelho e registra o recado deixado pela pessoa que fez a chamada telefônica.
b. *Esperando*: a secretária está ociosa esperando ser ativada.
c. *Revendo recados*: algum usuário da secretária requisitou que o aparelho apresentasse os recados gravados.

9-7: Considere o DTE para a classe `OfertaDisciplina` apresentado na Seção 9.5. Suponha que as ofertas de disciplina tenham também um número mínimo de alunos para que possam ser fechadas. Ou seja, se esse número mínimo for N, não pode haver ofertas de disciplina com uma quantidade de alunos inscritos menor que N. Modifique o DTE de `OfertaDisciplina` para contemplar essa nova restrição.